already hold info about which points are in the margin area between each pair of adjacent shards).
Each reducer will output a list of clusters to merge, and a list of outlier points to keep for each pair of shards.

#6 Before returning, we need to combine the results from reducers by merging clusters (possibly re-indexing clusters in a global way) and fixing the list of outliers.

Reducers need to receive, from each mapper, the list of clusters found, the list of noise points (if any), and the list of points in the margin region between the two shards that the reducer will process.

The reducer then checks if there is a core point in the shared margin region, and if it is so, it merges the two clusters to which the points belongs: in the example in figure 13.13 we deliberately used a global incremental indexing for clusters at this stage, but in reality this global indexing is not easily achievable with a single pass! That's because we don't know in advance how many clusters a mapper will find. Merging clusters can also be handled locally, but at some point an extra step will be needed to re-index all clusters globally.

If we suppose that clusters have global indexing, however, we can handle merging clusters using a data structure with which you should already be familiar: the *disjoint set*, that we described in chapter 5.

You might have noticed that there are some edge cases we should keep in mind: clusters in one shard, for instance, can be subsets of a bigger cluster in another shard; in the example, reducer 2 gets clusters C_1 and C_6, with the latter completely included in the former: in this case, even if no